



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Concurrent Server TCP/IP

Concurrent Real Time Programming

Furlan Davide

July 20, 2026

Problem Statement: The Real-Time Challenge



- **Objective:** Simulate a system of periodic tasks with strict timing constraints: Execution Time (C_i), Period (P_i), and Deadline (D_i).
- **The Constraint:** In a hard real-time context, adding a new task to a running system is not guaranteed to be safe.
- **Dynamic Environment:** The system must respond to asynchronous client requests to activate or deactivate tasks via a TCP/IP interface.
- **Mathematical Necessity:** Before any task activation, the supervisor must prove that the current task set remains *schedulable* under Rate Monotonic assumptions.

System Architecture: High-Level View



The system is divided into three distinct functional layers:

1. Communication Layer (Networked I/O)

Asynchronous TCP/IP socket management using `select()` to handle multiple client commands without blocking the main server loop.

2. Supervisor Layer (Control Logic)

A multi-threaded engine that manages a client pool and validates scheduling feasibility using a two-stage analytical approach (Utilization $\rightarrow$ RTA).

3. Execution Layer (Real-Time Tasks)

A pool of dedicated worker threads that execute periodic workloads with high-precision monotonic timing.

The Networked Interface: Client-Server Protocol



- **Protocol:** Client-Server communication over TCP/IP.
- **Message Framing:** To prevent partial reads/writes, a length-prefixed protocol is implemented:
 1. Server/Client sends an unsigned `int netLen` (Network Byte Order via `htonl`).
 2. The actual payload of length `len` follows immediately.
- **Command Set:**
 - `ACTIVATE [ID]`: Request task startup.
 - `BLOCK [ID]`: Request task deactivation.
 - `STOP / QUIT`: Management of connection/server lifecycle.

Concurrency Model: Client Management



- **The Multi-Client Problem:** The server must handle N simultaneous clients without allowing one slow client to block others.
- **Implementation:**
 - **Main Loop:** Uses `select()` to monitor the `server_socket` for new connections.
 - **Thread-per-Client:** Upon `accept()`, a new thread (`connectionHandler`) is spawned.
 - **Client Buffer:** A fixed-size array of `Client` structures tracks metadata (IP, Port, Thread ID, Socket FD).
- **Throttling:** A semaphore (`free_slots_sem`) and condition variable (`roomAvailable`) ensure the server does not exceed its maximum thread capacity.

Concurrency Model: Task Execution



- **The Thread-per-Task Model:** Once a task is deemed "schedulable," a specialized worker thread is spawned.
- **The Task Lifecycle:**
 1. **Validation:** Check RTA results.
 2. **Instantiation:** Assign unique `instance_id` and slot in `tasks_active`.
 3. **Execution:** The thread enters a periodic loop.
 4. **Deactivation:** The thread monitors its active flag; once set to 0, it exits cleanly.
- **Detachment:** Threads are `pthread_detach`'d to prevent zombie processes and ensure automatic resource reclamation.

Synchronization: Protecting Shared State



In a multi-threaded environment, race conditions during RTA analysis are fatal. We employ several mutexes:

- **active_tasks_mutex:** The most critical lock. It protects the `tasks_active` array. It is held during:
 - Task addition/removal.
 - The entire duration of the RTA calculation (to ensure a consistent snapshot of the system).
- **mutex (Client Buffer):** Protects the integrity of the `clients[]` array when clients connect or disconnect.
- **mutexstopserver:** Protects the global shutdown flag to ensure all threads see the signal simultaneously.

Synchronization: Throttling and Signaling



- **Semaphores (`sem_t`):**
 - `free_slots_sem`: Acts as a counting semaphore to limit the number of active tasks. This prevents system exhaustion.
- **Condition Variables (`pthread_cond_t`):**
 - `roomAvailable`: Used in the `addclient` function. If the client buffer is full, the thread waits on this variable, preventing busy-waiting and allowing efficient CPU usage.
- **Atomicity:** The use of these primitives ensures that the transition between "Task Requested" and "Task Running" is atomic and thread-safe.

Scheduling Theory: Rate Monotonic Scheduling



- **Core Principle:** We assume a fixed-priority, preemptive scheduler where priority is inversely proportional to the task period (P_i).
- **The Priority Logic:**

$$\text{If } P_i < P_j \implies \text{Priority}(i) > \text{Priority}(j)$$

- **Implementation:**
 - During RTA, the system collects all currently active tasks.
 - It applies `qsort` using the `compare_rms_priority` function.
 - This sorting is essential because the RTA calculation depends on the interference from higher-priority tasks.

Schedulability: The Utilization Factor



Before performing the heavy-duty RTA, we perform a fast-path check using the **Utilization Factor** (U):

The Mathematical Test

$$U = \sum_{i=1}^n \frac{C_i}{P_i}$$

- **Case 1:** $U > 1.0$: The system is physically over-utilized. Immediate rejection.
- **Case 2:** $U \leq 0.693$ (**approx. $\ln 2$**): According to the Liu and Layland bound, the task set is guaranteed to be schedulable under RMS.
- **Case 3:** $0.693 < U \leq 1.0$: The system is in the "uncertain zone." We must proceed to the iterative RTA to find the exact response time.

Schedulability: Iterative RTA Algorithm



For tasks in the "uncertain zone," we calculate the worst-case response time (R_i) for each task.

The Iterative Formula

$$R_i^{(k+1)} = C_i + \sum_{j \in HP(i)} \left\lceil \frac{R_i^{(k)}}{P_j} \right\rceil C_j$$

- $HP(i)$: The set of tasks with higher priority than task i .
- C_j, P_j : Execution time and period of higher-priority tasks.
- $\lceil \dots \rceil$: The ceiling function, representing the number of times a higher-priority task preempts task i .
- **Termination:** The iteration continues until $R_i^{(k+1)} = R_i^{(k)}$. If $R_i > P_i$ at any point, the task is *unschedulable*.

- **Data Integrity:** We use a local `eval_set` array during RTA to avoid holding the `active_tasks_mutex` for an excessive amount of time, though the initial copy must be protected.
- **Precision:** We utilize `long double` and the `ceil` function to prevent floating-point rounding errors that could lead to false "schedulable" results.
- **Code Logic Snippet:**

- **Resource Leak Prevention:**

- Using `pthread_detach` ensures that when a task is "blocked," its thread resources are reclaimed without needing an explicit `pthread_join` in the main loop.
- The `find_instance_to_deactivate` logic ensures we target the correct task instance, preventing accidental deactivation of newly arrived tasks of the same type.

- **Error Handling:**

- Robust error reporting using `strerror(errno)` for socket failures.
- Graceful shutdown via a broadcast mechanism that signals all clients to close.

- **Complexity Managed:** Successfully combined Network I/O, Multi-threading, and Real-Time Scheduling into a cohesive C system.
- **Determinism:** The implementation of RTA provides a mathematical guarantee of system stability, transforming a "best-effort" server into a "real-time" supervisor.
- **Concurrency:** Robust use of POSIX primitives (Mutex, Semaphore, CV) ensures data integrity under high contention.
- **Summary:** The project demonstrates the rigorous requirements of systems programming where timing, synchronization, and mathematical validation are paramount.